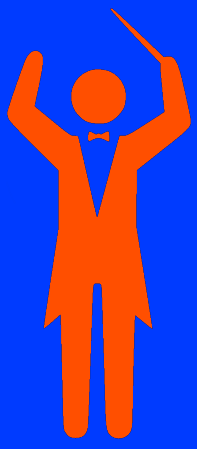
{EPITECH}

BERNSTEIN_

< ORCHESTRATE YOUR OWN SYMPHONY OF CONTAINERS />



BERNSTEIN

Orchestration is the *arrangement of a piece of music in parts so that it can be played by an orchestra* (from the *Oxford Advanced Learner's Dictionary*).

In other terms, it is the process of organizing the different components of a system in order for them to be (correctly) executed by a generic infrastructure.

Leonard Bernstein was an exceptional composer (his most famous work being the soundtrack of *West Side Story*), pianist, and conductor in his days.



Leonard Bernstein conducting Hector Berlioz's *Symphonie fantastique, Op. 14* with the Orchestre National de France in 1976.

During this project, you are going to become the Leonard Bernstein of containers!

The orchestra you are going to conduct is one of the most popular of its kind: Kubernetes.

Kubernetes is an open-source platform that allows you to manage containerized applications and services, in a scalable and automatic way.

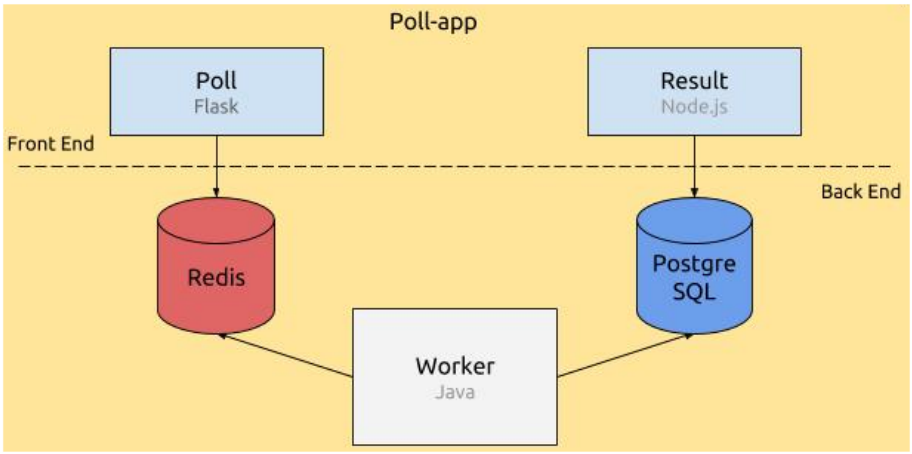
{EPITECH}

The *Poll* application

A simple web application, the *Poll* application, is used as the basis of this project.

There are five elements constituting it:

- ✓ ***Poll***, a Flask Python web application that gathers votes and push them into a Redis queue.
- ✓ A **Redis queue**, which holds the votes sent by the Poll application, awaiting for them to be consumed by the Worker.
- ✓ The ***Worker***, a Java application which consumes the votes being in the Redis queue, and stores them into a PostgreSQL database.
- ✓ A **PostgreSQL database**, which (persistently) stores the votes stored by the Worker.
- ✓ ***Result***, a Node.js web application that fetches the votes from the database and displays the... well, result. ;)





This app looks familiar, does it not? ;)

Both the source code and the Dockerfiles are supplied.

Goal of the project

The goal is to set up a complete deployment process for the *Poll* application, from the source code to the Kubernetes cluster.

It must, in order:

- ✓ Build the various application images;
- ✓ Push the images to a **private** Docker registry;
- ✓ Deploy the application to a Kubernetes cluster.

After the deployment, both the *Poll* and *Result* pages must be accessible from the outside world.

The way you decide to implement the deployment process is entirely up to you, including the eventual use of supplementary tools.



The final expected delivery is the entire set of scripts and files needed to deploy the application, **except for**:

- ✓ the credentials/API keys for the cloud platform you chose to use;
- ✓ and the `POSTGRES_PASSWORD` secret.



Even if the above exceptions are not included in the final delivery, you need to document how and where to set them for a deployment.

Provided that they have a ready-to-use Kubernetes cluster on the same platform you chose to use, the evaluator must be able to clone your repository, set the necessary credentials, and launch the deployment from scratch by themselves.

Cloud platforms

It is **highly** recommended to use a ”Kubernetes-as-a-Service” platform.
Examples of such platforms include (but are not limited to):

- ✓ Azure Kubernetes Service;
- ✓ Amazon Elastic Kubernetes Service;
- ✓ Google Kubernetes Engine;
- ✓ Digital Ocean.

See also the cloud platforms there: `https://education.github.com/pack`.

Installing a full Kubernetes cluster alongside a Docker registry locally is complex, so really have a look at cloud platforms that offer Kubernetes (and Docker registries) as a service.



As an Epitech student, you have access to credits on Azure, go check it out!



Make sure to not waste your credits, as you might need them for future projects.
Cost management is a crucial skill for any DevOps engineer.

Evaluation

The project is entirely evaluated through a defense.

You will need to demonstrate several points.

Initial deployment

Starting from a fresh cluster, you will need to demonstrate the deployment of the application using your delivered deployment resources.

After the deployment, the application must be accessible from the outside world, the evaluator must be able to vote on the *Poll* page, and successfully see the result on the *Result* page.



You will be allowed to check what the external IP addresses of the deployed application are and give them to the evaluator.

No-downtime update

After the successful initial deployment, the source code of one of the application will be changed and will need to be redeployed to the cluster.

You will need to demonstrate the ability to deploy an update **without any downtime**.



Without any downtime means that continously requesting the application should not result in any error.



Technical choices justification

There are many many possible ways to design the deployment process and its implementation, as well as the configuration of the various tools involved (including the Kubernetes cluster itself).

The various decisions you made, or decided *not* to make, will be discussed in detail, and you will need to justify them.

This part will not be related to the fundamental features expected for the application to work (these being evaluated in the previous points), but rather to what can be described as *best practices*.



That obviously means that you will need to decide what to do, and what *not* to do, very conciously.
Having a deployment that *works* is not enough! Far from it!
Making it resilient, scalable, and easy to operate is extremely important.



In order to know what the different options are for you at all times, you will obviously need to thoroughly read the documentation of the various tools you are going to use.

Technical mastery

Doing the project in the right way will ensure that you have a good understanding of the various parts involved, both theoretically and practically.

You must be able to answer various questions about your deployment, including (but not limited to):

- ✓ *What will happen if we do X?* (e.g., "What will happen if we use `kubectl drain` on this node?")
- ✓ *How would we do X?* (e.g., "How would we limit the processing power of Redis?")



This point is obviously related to the previous one, so the same tips apply here.

v 2

